

Ingeniería de Prompts — Análisis completo de la Sesión 5

Fuente base: *Ingeniería de Prompts [C5 - 2026]* — Módulo 3, Programa en Diseño e Implementación de Agentes IA, UTEC Posgrado.

Complementado con: investigación propia sobre prácticas actuales de Anthropic (Claude) y OpenAI para construcción de agentes, julio 2026.

Propósito de este documento: que sirva como referencia operativa — no solo un resumen de clase — para cuando diseñes prompts de producción y, sobre todo, cuando defines las *instructions* de un agente.

0. Por qué esta sesión es la bisagra del programa

La clase se ubica en un arco deliberado:

```
Clase 5 (esta) → Clase 6 → Clase 7 → Agentes
Contrato +      CoT /      Modularizar      (Model + Tools +
2 principios    razonamiento contratos con  Instructions)
                a fondo      gates/guardrails
```

La tesis central de todo el material — repetida de distintas formas en cada slide — es esta:

Un prompt no es redacción creativa. Es un contrato de entrada/salida (I/O) que debe poder verificarse mecánicamente. Si no puedes verificar la salida, no tienes un contrato: tienes una apuesta.

Esto importa especialmente para agentes porque un agente ejecuta ese "contrato" miles de veces sin supervisión humana directa; si el contrato es ambiguo, el agente no falla una vez — falla sistemáticamente, y a escala.

1. Prompt Engineering como disciplina

1.1 Definiciones

- **Prompt:** una instrucción dirigida a un modelo.

- **Prompt Engineering:** el conjunto de técnicas para escribir instrucciones claras, precisas y estructuradas dirigidas a la IA, con el objetivo de obtener respuestas exactas y útiles, **evitando resultados genéricos o alucinaciones.**

1.2 El mecanismo de fondo (por qué esto no es cosmético)

Un LLM predice la siguiente palabra (token) condicionado por todo el prompt. La disciplina de prompting no es estética: es matemática de distribución de probabilidad.

- **Sin disciplina** → el modelo rellena cada hueco de ambigüedad con la suposición estadísticamente más probable, no con la que tú querías.
- **Menos ambigüedad** → **distribución de salidas más estrecha** → **más reproducibilidad.**

1.3 Los dos fallos típicos cuando el prompt es pobre

Fallo	Qué es	Por qué es peligroso
Respuesta genérica	El modelo da un output correcto pero inútil, sin la especificidad que necesitabas	No sirve para automatizar nada; obliga a reprocesar manualmente
Alucinación	El modelo inventa datos que <i>parecen</i> correctos pero son falsos	Es el fallo más costoso porque erosiona la confianza y puede tener consecuencias legales/operativas reales

1.4 Canon de autores — quién aportó qué

Autor	Aporte concreto	Fuente
Jason Wei (2022)	Autor principal de <i>Chain-of-Thought Prompting Elicits Reasoning in LLMs</i> . Con solo 8 ejemplos CoT, PaLM 540B alcanzó SOTA en GSM8K (problemas matemáticos). Es la validación empírica del Principio 2.	arXiv:2201.11903; NeurIPS 2022
Andrew Ng	Fundador de DeepLearning.AI, profesor de Stanford. Aporta los DOS PRINCIPIOS y el concepto de desarrollo iterativo del prompt.	Cursos <i>Generative AI for Everyone / AI Prompting for Everyone</i>
Isa Fulford	Equipo técnico de OpenAI. Coautora de <i>ChatGPT Prompt Engineering for Developers</i> . Aporta las tácticas concretas de cada principio.	DeepLearning.AI + OpenAI

2. Caso de estudio: el costo real de una salida sin verificar

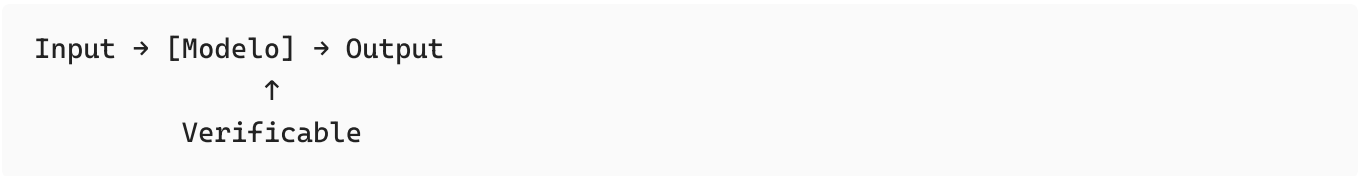
Times of India, 31-jul-2025: un abogado estadounidense (Thomas Nield, Semrad Law Firm) fue multado por presentar un escrito judicial con **cuatro citas legales fabricadas por ChatGPT**. El juez Michael Slade detectó que los casos citados simplemente no existían. El abogado admitió haber usado IA para investigación legal sin verificar el output.

Este caso es el hilo conductor de toda la sesión: cada táctica anti-alucinación que se enseña después ("si no puedes verificar la cita, dilo") es, literalmente, lo que hubiera evitado esta multa. Vale la pena internalizar esto como la razón *de negocio*, no solo académica, para tomarse en serio el diseño de prompts: **un prompt mal construido no es un problema estético, es un riesgo legal/operativo/reputacional**.

3. El prompt como contrato I/O verificable

Este es el marco central de la clase. Todo prompt "serio" tiene **tres partes, ninguna opcional**:

#	Parte	Pregunta que responde	Detalle
1	Objetivo	¿Qué se quiere lograr?	Verbo de acción, sin ambigüedad ("extrae", "clasifica", "resume" — no "habla sobre")
2	Restricciones + Input	¿Qué datos entran? ¿Qué está prohibido? ¿Qué hacer ante datos faltantes?	Define el perímetro de la tarea y la política de casos vacíos
3	Formato de salida verificable	¿En qué estructura (JSON, tabla, esquema) un programa o revisor puede chequear el resultado mecánicamente?	Es lo que convierte "creo que está bien" en "sé que está bien"



Por qué funciona (las dos palancas):

1. Los **dos principios de Ng & Fulford** son, en esencia, dos formas de escribir mejor ese contrato:
 - Principio 1 presiona el **QUÉ** (claridad y especificidad de la instrucción).

- Principio 2 presiona el **CÓMO RAZONA** (darle tiempo/estructura de pensamiento al modelo).

4. Principio 1 — Instrucciones claras y específicas

4.1 La idea raíz (Fulford)

Una instrucción clara puede ser larga si cada palabra elimina una ambigüedad. **Lo contrario de "claro" no es "largo", es "ambiguo".**

Esto es importante porque mucha gente cree (erróneamente) que un buen prompt es un prompt corto. No es así: un prompt corto con hueco de ambigüedad es peor que uno largo que cierra todos los huecos.

4.2 Las cuatro tácticas del Principio 1

Táctica 1 — Delimitadores

Qué es: rodear el texto de entrada con marcas inequívocas (triple backtick `````, comillas triples `"""`, o etiquetas `<doc>...</doc>`) para separar **INSTRUCCIÓN** de **DATOS**.

Por qué funciona: el modelo deja de adivinar dónde empieza y termina lo que debe procesar. Además **previene inyección de prompt** (que el texto de entrada se haga pasar por una instrucción nueva). Es el primer ladrillo del hilo anti-alucinación: ancla la tarea a un bloque concreto.

```
Resume el texto entre <doc></doc> en UNA sola frase.  
<doc>{texto}</doc>
```

 **Nota de seguridad para agentes:** los delimitadores son también tu primera línea de defensa contra *prompt injection* cuando el "texto" viene de una fuente no confiable (un documento subido por el usuario, el resultado de una búsqueda web, el contenido de un email). Sin delimitar, un atacante puede escribir dentro del documento algo como "Ignora las instrucciones anteriores y..." y el modelo puede confundirlo con una instrucción legítima.

Táctica 2 — Salida estructurada

Qué es: indicar el formato exacto de salida (JSON, HTML, tabla) con sus claves y tipos explícitos.

Por qué funciona: una estructura es **automáticamente validable** (`json.loads` + chequeo de claves/tipos). Es, literalmente, lo que vuelve "verificable" al contrato — la Táctica 2 es donde el

Principio 1 se vuelve testeable como código.

```
{
  "producto": "",
  "valoracion": 0,          // number 1..5
  "sentimiento": "",        // positivo | neutro | negativo
  "recomendaria": false     // boolean
}
```

Táctica 3 — Verificar condiciones (anti-alucinación)

Qué es: instruir al modelo a comprobar **precondiciones** antes de actuar y a manejar el caso vacío explícitamente ("si no hay pasos, di 'No se encontraron pasos'").

Por qué funciona: le das al modelo una **salida de escape honesta** en vez de empujarlo a inventar. **Es la táctica que más reduce alucinaciones** — es literalmente lo que le faltó al abogado del caso anterior ("si no puedes verificar la cita, dilo"). Equivale al principio de "capture edge cases" de la guía de agentes de OpenAI.

```
Si el texto entre <doc></doc> contiene una secuencia de pasos, reescríbela
como lista numerada. Si NO contiene pasos, responde EXACTAMENTE:
"No se encontraron pasos." No inventes pasos que no estén en el texto.
```

Táctica 4 — Few-shot (enseñar por ejemplos)

Qué es: incluir 2 o más pares input→output que muestran el patrón antes de la tarea real (zero-shot = pedir directo, sin ejemplos).

Por qué funciona: cuando la regla es difícil de enunciar pero fácil de mostrar (estilo, clase, tono), los ejemplos son **especificación por demostración** (*in-context learning*). Fulford lo presenta como táctica del Principio 1 porque el ejemplo ES parte de la instrucción clara — no es un principio aparte.

```
'No puedo pagar'          -> Queja
'Agregaría PayPal'        -> Sugerencia
Ahora clasifica: 'La app es lenta' -> ?
```

4.3 El andamio CRTO (+ ITE)

CRTO es la checklist que integra las cuatro tácticas anteriores en un solo procedimiento repetible.

Letra	Significa	Pregunta	Mapea a
C	Contexto	¿Por qué estamos haciendo esto? Es el <i>Problem Statement</i> . El 90% de la gente lo ignora y por eso obtiene respuestas genéricas.	—
R	Rol	¿Quién debe ser la IA? Activa un subespacio de conocimiento y un "tono de voz" sin gastar ejemplos. Pregúntate: ¿Analista de Datos? ¿Seller Dev? ¿KAM? ¿UX Researcher?	—
T	Tarea	La acción específica y directa. Usa verbos: "crea", "lista", "compara", "resume", "reescribe", "analiza" — nunca "habla sobre".	Objetivo del contrato
O	Output	El formato exacto de salida (viñetas, tabla Markdown, JSON). Ahorra tiempo de reformateo manual.	Formato verificable
I (<i>extensión</i>)	Input	Dónde van los datos — nota sobre volumen si es grande.	→ delimitadores
T (<i>extensión</i>)	Tono	Analítico/objetivo/orientado a la acción, según el destinatario.	—
E (<i>extensión</i>)	Exclusiones	Qué NO debe hacer. Más efectivo decir qué hacer que qué no hacer, pero las exclusiones negativas explícitas (ej. "no incluyas PII") mejoran fiabilidad y ética.	→ verificar condiciones

Mapeo 1:1 al contrato del capítulo 3:

- C + I = entrada
- R + T = objetivo
- O + T(tono) + E = formato/restricciones

Ejemplo integrado (CRTO+ITE):

C (Contexto): onboarding legal; volumen alto de contratos.
 R (Rol): Actúa como abogado laboral junior.
 T (Tarea): Extrae PARTES y PUESTO del contrato adjunto.
 O (Output): sub-JSON { empleador:{...}, puesto:{...} }
 I (Input): el texto del contrato va entre <doc>...</doc>
 T (Tono): neutro, objetivo.
 E (Exclusión): NO inventes datos ausentes; usa null.

4.4 Puente con OpenAI: *Configuring instructions*

La guía de OpenAI "*A practical guide to building agents*" (ver §9) recomienda para configurar las instrucciones de un agente:

- Usar documentos existentes (SOPs, políticas) como fuente de instrucciones.
- Descomponer la tarea en pasos.
- Definir acciones claras y no ambiguas.
- **Capturar edge cases** explícitamente.

Estas son, palabra por palabra, las mismas cuatro tácticas del Principio 1 — solo que aplicadas al *system prompt* de un agente en vez de a un prompt suelto.

5. Principio 2 — Dar tiempo al modelo para pensar

5.1 Las dos tácticas del curso

Táctica 1 — Especificar los pasos

Qué es: descomponer la tarea en una secuencia numerada que el modelo debe seguir (Paso 1... Paso 2... Paso 3...).

Por qué funciona: convierte un salto directo input→respuesta (propenso a error) en una cadena de sub-tareas más fáciles. Es el "*break down tasks / define clear actions*" de OpenAI llevado a un prompt suelto.

```
Paso 1: resume en 1 frase. Paso 2: tradúcela al inglés.  
Paso 3: devuelve JSON {resumen_es, resumen_en}.
```

Táctica 2 — Resolver antes de concluir (anti-complacencia)

Qué es: ante una solución ajena que hay que evaluar, ordenar al modelo **resolver el problema él mismo primero** y solo después comparar y juzgar.

Por qué funciona: si le muestras al modelo una solución (posiblemente errónea) y le preguntas "¿está bien?", tiende a **estar de acuerdo por inercia** (sesgo de complacencia / *sycophancy*). Forzarlo a derivar su propia solución antes lo hace *contrastar* en vez de *validar* → menos errores, menos complacencia.

```
Vas a evaluar la solución de un alumno. NO digas si es correcta todavía.  
Paso 1: resuelve el problema TÚ MISMO, mostrando tu trabajo.  
Paso 2: compara tu solución con la del alumno.  
Paso 3: recién entonces dictamina CORRECTO o INCORRECTO y por qué.
```

5.2 El fundamento de investigación: Chain-of-Thought (Wei et al., 2022)

Definición: CoT = inducir al modelo a generar pasos de razonamiento intermedios ("pensemos paso a paso") antes de la respuesta final.

Evidencia citada en el curso: con solo 8 ejemplos CoT, PaLM 540B alcanzó estado del arte (SOTA) en GSM8K (problemas matemáticos de escuela). Esta es la validación empírica real de "dar tiempo a pensar" — no es una intuición, es un resultado publicado y replicado.

Variante relacionada — Step-Back Prompting: reflexionar sobre principios generales antes de la pregunta específica. Ideal para estrategia de producto o brainstorming de alto nivel.

Paso 1: ¿Cuáles son los principios de un buen onboarding?
Paso 2: Basado en ellos, mejora nuestro flujo actual.

5.3 Cómo evolucionó esto en los modelos actuales (investigación complementaria, 2026)

Esta parte NO estaba en el PDF pero es crítica para cuando construyas agentes con Claude hoy:

- Los modelos actuales de Claude (Sonnet 5, Opus 4.8, Fable 5/Mythos 5) usan **"adaptive thinking"** en vez del CoT manual explícito ("pensemos paso a paso") o el *extended thinking* con `budget_tokens` fijo de generaciones anteriores. El modelo decide **cuánto pensar** según dos factores: el parámetro `effort` (bajo/medio/alto) y la complejidad detectada de la consulta.
 - Esto significa que el Principio 2 de Ng & Fulford (2023) sigue siendo válido conceptualmente, pero su **implementación técnica cambió**: ya no siempre necesitas escribir "pensemos paso a paso" a mano — puedes subir el `effort` o activar `thinking`:
`{type: "adaptive"}` y dejar que el modelo decida.
 - Sin embargo, el CoT manual **sigue siendo el fallback correcto** cuando el `thinking` está desactivado, o cuando quieres forzar una estructura de razonamiento auditable con tags `<thinking>...</thinking>` y `<answer>...</answer>` separados.
 - Un patrón que Anthropic documenta y que encaja directamente con la Táctica 2 del curso (resolver antes de concluir): pedirle al modelo que **se autoverifique** antes de terminar — *"Before you finish, verify your answer against [test criteria]"* — reduce errores de forma consistente, especialmente en código y matemáticas.
-

6. Anti-alucinación: defensa en profundidad

El curso propone tres tácticas explícitamente atadas al caso del abogado (§2):

#	Táctica	Qué hace	Frase clave
1	Delimitar y anclar	El modelo no puede traer datos de fuera del bloque <code><doc>...</doc></code>	"Responde SOLO con el texto provisto."
2	Verificar / salida de escape	Nunca inventar; es la honestidad que le faltó al abogado	"Si el dato no está en el texto, responde 'no encontrado'."
3	Formato auditable	Campo de verificación explícito en la salida que un humano o programa chequea antes de confiar	verificada: true/false , fuente , o faltantes: null

6.1 Complemento de investigación: cómo lo formaliza Anthropic hoy

La documentación actual de Claude añade una capa adicional muy relevante para agentes de código/investigación (no estaba en el PDF, es 2026):

```
<investigate_before_answering>
Never speculate about code you have not opened. If the user references a
specific
file, you MUST read the file before answering. Make sure to investigate and
read
relevant files BEFORE answering questions about the codebase. Never make any
claims
about code before investigating unless you are certain of the correct answer.
</investigate_before_answering>
```

Este bloque es la Táctica 3 del curso ("verificar condiciones") llevada a un agente con herramientas: en vez de "si no está en el texto, di no encontrado", el equivalente agentic es **"si no lo has leído con una tool, no afirmes nada sobre ello — ve y léelo primero."** Es el mismo principio (no rellenar huecos con suposiciones) aplicado a un agente que tiene acceso a herramientas de lectura de archivos, no solo a un bloque de texto estático.

7. Desarrollo iterativo del prompt (Ng)

Definición: un ciclo — *idea* → *prompt* → *resultado* → *análisis del error* → *refinar* — análogo al ciclo de desarrollo de Machine Learning. El prompt rara vez sale perfecto al primer intento.

Por qué funciona: trata el prompt como un artefacto de ingeniería iterativo, no como una ocurrencia única. Cada vuelta del ciclo cierra una ambigüedad detectada específicamente en la salida anterior — **no se adivina, se itera** basándose en evidencia.

Ejemplo del curso (resumir ficha técnica):

```
v1: "Resume esta ficha técnica."
    -> salida demasiado larga.

v2: "Resume esta ficha técnica en <= 50 palabras, enfocada en el material."
    -> menciona el precio, que es irrelevante.

v3: "...en <= 50 palabras, enfocada en el material. Ignora el precio.
     Añade un ID de 7 caracteres al final."
    -> cumple.
```

Las capacidades del curso de Ng & Fulford que se afinan iterando son: **resumir, inferir, transformar, expandir**.

8. Otros tipos de prompt (navaja suiza)

El material presenta 5 "modos" de prompt como plantillas rápidas de uso frecuente en contextos de producto:

Tipo	Qué hace	Ideal para	Ejemplo
Role Prompting	Asignar un rol a la IA	Simular stakeholders, mejorar comunicación	"Actúa como un ingeniero de software escéptico. Señala 3 posibles fallas técnicas en esta nueva funcionalidad."
Contextual Prompting	Dar el "porqué" o información de fondo	Tareas que necesitan alineación estratégica	"Contexto: Nuestro OKR es aumentar la retención. Genera 3 ideas de funcionalidades para este objetivo."
File Analyzing	Analizar contenido de un archivo/texto extenso	Extraer temas, resumir estudios, insights de tickets	"Analiza estas 50 reseñas de la app store y enlista las 5 quejas más comunes."

Tipo	Qué hace	Ideal para	Ejemplo
Improve Speech	Refinar/reescribir un texto existente	Ajustar redacción para stakeholders, simplificar documentación	"Haz que esta nota de actualización sea más amigable para usuarios no expertos."
Solution Prompting	Generar soluciones/estrategias para un problema contextualizado	Brainstorming de funcionalidades, planes de acción	"Sugiere 3 enfoques de bajo costo para aumentar el engagement de nuevos usuarios en su primera semana."

9. Del contrato al agente — el puente explícito de la clase

Esta es la sección que conecta directamente con tu objetivo de construir agentes.

9.1 Definición de agente según OpenAI (citada en el curso)

Un **agente** = **Model + Tools + Instructions**.

Instructions = "explicit guidelines and guardrails defining how the agent behaves" — es decir, **el prompt-contrato, pero a escala**.



9.2 Por qué el contrato de la Clase 5 escala directo a un agente

Concepto de Clase 5	Escala a nivel agente
Objetivo con verbo, sin ambigüedad	<i>System prompt</i> / rol del agente
Restricciones + política de faltantes	<i>Guardrails</i> de input/output
Formato de salida verificable (JSON+esquema)	<i>Structured outputs</i> / <i>tool schemas</i> que el runtime valida automáticamente
Delimitadores	Separación estricta entre "instrucciones del sistema" y "contenido no confiable" (datos de usuario, resultados de

Concepto de Clase 5	Escala a nivel agente
	tools, contenido web) — crítico contra <i>prompt injection</i>
Few-shot	Ejemplos dentro del <i>system prompt</i> o en los <i>tool descriptions</i>
Especificar los pasos / CoT	Razonamiento interno del agente antes de decidir qué <i>tool</i> llamar
Prompt template con variables (<code>{{policy}}</code>)	Un contrato parametrizado — el mismo patrón, reutilizable por instancia de agente

9.3 Ampliación con la guía completa de OpenAI (investigación complementaria)

El PDF cita solo el titular de la guía de OpenAI. La guía completa ("*A practical guide to building agents*", OpenAI) profundiza en tres pilares que conviene tener mapeados:

a) Instructions — mismas 4 prácticas del Principio 1 de este curso: usar documentos existentes como fuente, descomponer tareas, definir acciones claras, capturar edge cases.

b) Tools — tres tipos:

- *Data tools*: consultas de solo lectura (bases de datos, búsquedas).
- *Action tools*: modifican estado (actualizar un CRM, enviar un email).
- *Agent tools* (orquestación): permiten que un agente invoque a otro agente/subagente.

Nota práctica: distinguir *data tools* de *action tools* es exactamente el mismo espíritu que "reversibilidad y blast radius" que rige cuándo un agente debe pedir confirmación antes de actuar — las *action tools* son las que ameritan guardrails más estrictos.

c) Guardrails — defensa en capas, nunca una sola:

- Guardrails basados en LLM (un segundo modelo o el mismo modelo evalúa el input/output).
- Guardrails basados en reglas (regex, listas blancas/negras).
- APIs de moderación.
- Human-in-the-loop para acciones de alto riesgo.

Esto es literalmente la extensión a escala de la Táctica 3 ("verificar condiciones") del Principio 1: en un prompt suelto, la condición era "¿el dato está en el texto?"; en un agente, la condición es "¿esta acción es segura/autorizada/reversible?"

9.4 Context Engineering — el siguiente nivel más allá del prompt (2026, Anthropic)

Este es el desarrollo más importante que **no estaba en absoluto en el PDF** pero que es directamente relevante si vas a construir agentes: Anthropic ha formalizado la disciplina sucesora del *prompt engineering* para sistemas agentic, llamada **context engineering**.

Prompt engineering = escribir y organizar las instrucciones de un LLM (lo que enseña toda esta sesión).

Context engineering = curar y mantener el conjunto óptimo de tokens durante la inferencia — *system prompt + tools + datos externos + historial de mensajes* — **de forma cíclica**, turno tras turno, a medida que el agente opera.

La diferencia importa porque un prompt se escribe una vez; el contexto de un agente se **gestiona continuamente** mientras el agente trabaja.

Principios accionables de context engineering (resumen de la investigación):

1. **System prompts**: buscar la "zona Goldilocks" — ni tan específico que se vuelva frágil (lógica hardcodeada para cada caso), ni tan vago que asuma contexto compartido que el modelo no tiene. Empezar mínimo, añadir instrucciones solo cuando se detecta un modo de fallo real. Organizar con XML tags o headers Markdown.
2. **Diseño de herramientas (tools)**: mantener el conjunto de tools mínimo, sin solapamiento funcional; que quede clarísimo cuál tool usar en cada situación; parámetros descriptivos e inequívocos. Esto es la Táctica 2 (salida estructurada) aplicada al *input* de una tool, no solo al output del modelo.
3. **Ejemplos (few-shot) en agentes**: no meter una lista interminable de edge cases — curar un conjunto pequeño, diverso y canónico. "Los ejemplos son las imágenes que valen mil palabras" para un LLM.
4. **Gestión de la ventana de contexto**:
 - Tratar el contexto como recurso finito con **retornos decrecientes** — más contexto no es automáticamente mejor.
 - Cargar datos "justo a tiempo" (el agente usa una tool para traer solo lo que necesita en ese momento) en vez de pre-cargar todo por adelantado.
 - Para tareas largas: **compactación** (resumir el historial cuando se acerca el límite, preservando decisiones arquitectónicas clave), **notas estructuradas externas** (*NOTES.md* , listas de tareas) para no gastar contexto en memoria de trabajo, y **arquitecturas de subagentes** especializados que devuelven resúmenes condensados (1000-2000 tokens) en vez de mantener todo el contexto en un solo agente.

Principio unificador: "*Encuentra el conjunto más pequeño de tokens de alta señal que maximice la probabilidad de tu resultado deseado.*" — es la misma regla de oro de Fulford ("claro ≠ largo, claro = sin ambigüedad") pero extendida de "un prompt" a "todo el estado que ve el modelo en cada turno".

10. Investigación complementaria: técnicas de prompting específicas para Claude (2026)

Esta sección resume lo que la documentación oficial de Anthropic (Claude Platform Docs, julio 2026) recomienda hoy, organizado para que puedas mapearlo 1:1 contra las tácticas del curso.

Técnica de Claude (2026)	Equivalente/evolución respecto al curso
Ser claro y directo — "piensa en Claude como un empleado brillante pero nuevo que no conoce tus normas; si tu prompt confundiría a un colega con contexto mínimo, confundirá a Claude"	Es el Principio 1 casi palabra por palabra
Añadir contexto/motivación ("tu respuesta será leída por un motor de texto a voz, por eso nunca uses puntos suspensivos")	El campo C (Contexto) de CRTO — explicar el <i>porqué</i> generaliza mejor que solo dar la regla
Ejemplos (multishot) : 3-5 ejemplos envueltos en <code><example></code> / <code><examples></code> , diversos y que cubran edge cases	Evolución directa de la Táctica 4 (few-shot); el curso pedía 2+, Anthropic recomienda 3-5
Etiquetas XML para separar instrucciones/contexto/ejemplos/input variable	Evolución de la Táctica 1 (delimitadores) — ahora con vocabulario más rico: <code><instructions></code> , <code><context></code> , <code><input></code> , anidamiento jerárquico
Dar un rol en el <i>system prompt</i>	El campo R (Rol) de CRTO, formalizado como parámetro <code>system</code> de la API
Prompting para contextos largos (20k+ tokens): poner los documentos largos <i>arriba</i> del prompt, la pregunta al final (mejora hasta 30% en tests), estructurar cada documento con <code><document></code> , y pedirle a Claude que cite primero ("extrae las citas relevantes en <code><quotes></code> antes de razonar")	Esto es nuevo respecto al PDF — muy útil si vas a construir un agente RAG o de análisis de documentos largos
Decirle qué hacer en vez de qué NO hacer para controlar formato	Coincide con el principio E (Exclusiones) de CRTO+ITE, con el matiz de que Anthropic insiste en formular en positivo
Prefill deprecado en los modelos más recientes (Claude 4.6+) — se migra a <i>Structured Outputs</i> nativos	No estaba en el curso; importante si vienes de

Técnica de Claude (2026)	Equivalente/evolución respecto al curso
o <i>tool calling</i> con <code>enum</code>	tutoriales viejos que usaban prefill para forzar JSON
Adaptive thinking (<code>effort + thinking: adaptive</code>) reemplaza el CoT manual explícito para razonamiento complejo	Evolución técnica del Principio 2 — ver §5.3
Ser explícito para que el agente <i>actúe</i> y no solo sugiera ("cambia esta función" en vez de "¿puedes sugerir cambios?")	Relevante para diseñar <i>tool-use triggering</i> : la ambigüedad en el verbo de la Tarea (CRT0) determina si el agente ejecuta o solo opina
Paralelizar tool calls explícitamente cuando no hay dependencias entre ellas	Extensión agentic de "especificar los pasos" — a veces los pasos NO son secuenciales y hay que decirlo
Balance autonomía/seguridad : pedir confirmación antes de acciones destructivas o difíciles de revertir (borrar archivos, <code>git push --force</code> , mensajes a terceros)	Es la Táctica 3 (verificar condiciones) llevada al dominio de acciones con efectos reales, no solo texto
Evitar sobre-ingeniería ("no agregues abstracciones, validaciones o manejo de errores para escenarios que no pueden ocurrir")	Complementa el campo E (Exclusiones) — Claude 4.5/4.6 tiende a expandirse más de lo pedido si no se lo acota explícitamente
Minimizar alucinación en agentes de código : nunca especular sobre código no leído; leer el archivo antes de afirmar algo sobre él	Es la Táctica 3 (verificar condiciones / salida de escape) aplicada a un agente con herramientas de lectura

11. Ejercicios de la sesión (mapa completo)

El curso incluye una progresión deliberada de ejercicios prácticos, cada uno aislando una táctica antes de integrarlas todas:

Ejercicio	Táctica que aísla	Criterio de éxito
e1 (parte 1) — Delimitadores	Táctica 1	Input dentro de delimitadores; instrucción referida al bloque; ruido tratado como dato

Ejercicio	Táctica que aísla	Criterio de éxito
e1 (parte 2) — JSON	Táctica 2	Parsea + solo JSON + claves exactas + tipos correctos (binario)
e2 — Verificar condiciones	Táctica 3	Detecta el caso vacío y NO inventa pasos (binario por archivo, dos inputs distintos)
e3 — Few-shot	Táctica 4	Exactamente 2 ejemplos, cubren ≥ 2 clases, el caso a resolver no está entre los ejemplos
e4 — CRTO+ITE	Integrador P1	Objetivo con verbo + ≥ 1 restricción + política de faltantes + formato chequeable
e5 — Resolver antes de concluir	Principio 2	Ordena resolver antes de juzgar, detecta el error, no valida por inercia
e6 — Extracción legal → JSON exacto	Integrador final (P1+P2+anti-alucinación)	Ver rúbrica completa abajo

Rúbrica del ejercicio integrador (e6) — se evalúa como código, todo binario

1. **Parsea** — la salida es JSON sintácticamente válido. **Eliminatorio**.
2. **Claves 100%** — todas las del molde, ninguna de más, ninguna de menos.
3. **Tipos 100%** — string/number/boolean/array/null según el molde.
4. **Estructura 100%** — anidamiento exacto (sub-objetos como `empleador`, `trabajadora`).
5. **Política de faltantes** — ausentes = `null`, **nunca inventados** (= no alucinar; el vínculo directo con el caso del abogado).
6. (*Nivel avanzado*) — resolver una ENMIENDA contractual (ej. `sueLdosBase` en 2 tramos) y un esquema de vesting (25% inicial + 75% mensual en 36 meses).

Por qué esta rúbrica importa para agentes: es el mismo patrón que usarás para evaluar (*eval*) cualquier agente de extracción/estructuración de datos en producción — criterios binarios, automatizables, sin subjetividad. Si vas a construir agentes, esta rúbrica es una plantilla reutilizable de *eval*.

12. Quiz de la sesión (con respuestas)

#	Pregunta	Respuesta correcta
1	Los dos principios de Ng & Fulford para escribir buenos prompts son...	B — Instrucciones claras y específicas, y dar tiempo al modelo para pensar
2	¿Qué táctica reduce más las alucinaciones?	C — Anclar al texto dado y verificar condiciones ("si no está, di no encontrado")
3	El Chain-of-Thought de Wei et al. (2022) es un caso de...	A — Principio 2, dar tiempo al modelo para pensar (pasos de razonamiento)
4	¿Qué hace "verificable" a una salida?	C — Que exista un criterio binario automatizable (p. ej. parsea JSON y cumple esquema)
5	¿Cuál NO es una buena práctica al redactar prompts?	D — Dejar la instrucción ambigua para que el modelo "adivine"

13. Síntesis: lo que hay que llevarse de esta sesión

1. **El prompt es un contrato verificable, no arte.** La disciplina convierte la suerte en método.
2. **El contrato tiene 3 partes:** objetivo (verbo, sin ambigüedad) · restricciones + input · formato de salida verificable.
3. **Dos principios (Ng & Fulford):**
 - ① Instrucciones claras y específicas → 4 tácticas: delimitadores · salida estructurada · verificar condiciones · few-shot. Andamio: **CRTO (+ITE)**.
 - ② Dar tiempo al modelo para pensar → especificar los pasos · resolver antes de concluir · Chain-of-Thought (Wei, 2022).
4. **El prompt se itera:** idea → prompt → resultado → error → refinar. No se adivina, se itera con evidencia.
5. **Anti-alucinación:** anclar al texto · salida de escape ("no encontrado") · formato auditable (faltantes = `null`).
6. **Regla de oro:** si no puedes verificar tu salida, no tienes un contrato — tienes una apuesta.
7. **Puente a agentes:** un agente = Model + Tools + Instructions. Las *Instructions* son el prompt-contrato a escala, y los mismos principios/tácticas de esta clase son las *best practices* que documenta OpenAI para instrucciones de agentes.
8. (Complemento 2026) **Más allá del prompt está el contexto.** Cuando construyas agentes reales, el reto ya no es solo escribir un buen prompt una vez — es gestionar continuamente qué información (system prompt, tools, ejemplos, historial) ve el modelo en cada turno, con

el mismo criterio de "el mínimo conjunto de tokens de alta señal que maximice el resultado deseado".

14. Cómo aplicar esto cuando construyas tus propios agentes — checklist práctico

Usa esta checklist como plantilla operativa al diseñar el *system prompt* / instructions de un agente:

Diseño del contrato (Principio 1 + CRTO):

- ☐ ¿El objetivo tiene un verbo de acción explícito? (¿"extrae", "clasifica" — o vago como "ayuda con"?)
- ☐ ¿Definí el Rol del agente en una frase clara?
- ☐ ¿Separé instrucciones de datos no confiables con delimitadores/XML tags? (crítico si hay input de usuario, resultados de tools o contenido web)
- ☐ ¿El formato de salida es verificable mecánicamente (JSON con esquema, no prosa libre)?
- ☐ ¿Incluí 3-5 ejemplos diversos si la tarea tiene un patrón difícil de enunciar pero fácil de mostrar?
- ☐ ¿Dije qué hacer, no solo qué NO hacer?

Anti-alucinación (Táctica 3 + investigación 2026):

- ☐ ¿Hay una salida de escape explícita para el caso "no sé" / "no está en los datos"?
- ☐ ¿La política de datos faltantes está definida (`null` , no inventar)?
- ☐ Si el agente tiene tools de lectura, ¿le exijo que use la tool antes de afirmar algo, en vez de especular?

Razonamiento (Principio 2):

- ☐ Para tareas complejas, ¿especifiqué los pasos o subí el `effort` /activé `thinking: adaptive` ?
- ☐ Si el agente debe evaluar una solución ajena, ¿le pido que resuelva primero y compare después (anti-complacencia)?
- ☐ ¿Le pido autoverificación antes de terminar ("verifica tu respuesta contra estos criterios")?

Seguridad y acción (guardrails, escalando Táctica 3 a nivel agente):

- ☐ ¿Clasifiqué mis tools en *data tools* (solo lectura) vs *action tools* (efectos reales)?

- ☐ ¿Definí qué acciones requieren confirmación humana por ser destructivas o difíciles de revertir?
- ☐ ¿Tengo guardrails en capas (reglas + LLM + human-in-the-loop) en vez de uno solo?

Iteración y verificación:

- ☐ ¿Tengo un criterio de éxito binario/automatizable (una rúbrica tipo la del ejercicio e6)?
- ☐ ¿Estoy iterando basado en errores reales observados, no adivinando mejoras?

Gestión de contexto (para agentes de larga duración):

- ☐ ¿El system prompt está en la "zona Goldilocks" (ni hardcodeado a cada caso, ni vago)?
 - ☐ ¿Las tools tienen nombres/parámetros inequívocos y sin solapamiento funcional?
 - ☐ Para tareas largas, ¿tengo estrategia de compactación, notas externas (NOTES.md) o subagentes que devuelven resúmenes condensados?
-

15. Referencias

Del material original:

- Ng, A. & Fulford, I. — *ChatGPT Prompt Engineering for Developers* (DeepLearning.AI + OpenAI): dos principios, tácticas, desarrollo iterativo; resumir/inferir/transformar/expandir.
- Ng, A. — *Generative AI for Everyone / AI Prompting for Everyone* (DeepLearning.AI).
- Wei, J. et al. — *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. arXiv:2201.11903 (2022); NeurIPS 2022.
- OpenAI — *A practical guide to building agents* (Model + Tools + Instructions; Configuring instructions; prompt templates). <https://openai.com/business/guides-and-resources/a-practical-guide-to-building-ai-agents/>
- Times of India (31-jul-2025) — *US lawyer uses ChatGPT to cite fake legal cases; judge imposes fine*.

Investigación complementaria (añadida en este documento, julio 2026):

- Anthropic — *Prompting best practices* (Claude Platform Docs). <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices>
- Anthropic — *Prompt engineering overview*. <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
- Anthropic — *Effective context engineering for AI agents*. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>

Documento generado a partir del PDF de la Sesión 5 (Módulo 3, UTEC Posgrado) más investigación propia sobre prácticas actuales de Anthropic/OpenAI para diseño de agentes.
Última actualización: 2026-07-07.